Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 6.2

по курсу «Компьютерные сети»

«Разработка веб-ориентировнного FTP- клиента»

Студент группы ИУ9-32Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

1. Форма ввода реквизитов доступа: - ftp-сервер; - ftp-логин; - ftp-пароль.
2. Консоль ftp команд, передача команд выполняется асинхронно.
3. Обновление состояния текущей дирректории асинхронно.
4. Визувально выводить отличие лирректории от файла.
5. Выводить пермишины на дирректорию или файл.
6. Фронтенд и бэкэнд на разных серверах запускается.
7. Проверяет преподаватель самостоятельно заходя на URL фронтенда.

2 Результаты

Исходный код программы представлен в листинге 1–3.

Листинг 1: Реализация файла main.go

```go
package main

import (
    "fmt"
    "log"
    "net/http"
    "strings"

    "github.com/gorilla/websocket"
    "github.com/jlaffaye/ftp"
)

var (
    ftpClient *ftp.ServerConn
    upgrader  = websocket.Upgrader{}
    clients   = make(map[*websocket.Conn]bool)
)

func main() {
    http.HandleFunc("/ws", handleWebSocket)

    // Serve static files from the "html" directory
    fs := http.FileServer(http.Dir("./html"))
    http.Handle("/", fs)

    log.Println("Server running on http://185.104.251.226:5421")
    log.Fatal(http.ListenAndServe(":5421", nil))
```

```
28 }
29
30 // WebSocket handler
31 func handleWebSocket(w http.ResponseWriter, r *http.Request) {
32   upgrader.CheckOrigin = func(r *http.Request) bool { return true }
33   conn, err := upgrader.Upgrade(w, r, nil)
34   if err != nil {
35     log.Println("Error upgrading to WebSocket:", err)
36     return
37   }
38   defer conn.Close()
39   clients[conn] = true
40   defer delete(clients, conn)
41
42   for {
43     var message map[string]string
44     if err := conn.ReadJSON(&message); err != nil {
45       log.Println("Error reading message:", err)
46       break
47     }
48
49     handleCommand(message, conn)
50   }
51 }
52
53 // FTP command handler
54 func handleCommand(msg map[string]string, conn *websocket.Conn) {
55   command := msg["command"]
56   args := strings.Fields(command)
57   switch args[0] {
58   case "connect":
59     server, user, password := msg["server"], msg["username"], msg["
      password"]
60     if !strings.Contains(server, ":") {
61       server += ":21"
62     }
63
64     var err error
65     ftpClient, err = ftp.Dial(server)
66     if err != nil {
67       sendError(conn, "Failed to connect to FTP server", err.Error())
68       return
69     }
70
71     err = ftpClient.Login(user, password)
72     if err != nil {
```

```
73          sendError(conn, "Failed to log in to FTP server", err.Error())
74          return
75        }
76
77        sendMessage(conn, "Connected to FTP server")
78
79      case "ls":
80        if ftpClient == nil {
81          sendError(conn, "Not connected to any FTP server", "")
82          return
83        }
84
85        entries, err := ftpClient.List("")
86        if err != nil {
87          sendError(conn, "Failed to list directory", err.Error())
88          return
89        }
90
91        files := []map[string]string{}
92        for _, entry := range entries {
93          files = append(files, map[string]string{
94            "name": entry.Name,
95            "type": func() string {
96              if entry.Type == ftp.EntryTypeFile {
97                return "file"
98              }
99              return "directory"
100           }(),
101         })
102       }
103
104       conn.WriteJSON(map[string]interface{}{
105         "success": true,
106         "files":   files,
107       })
108
109     case "rm":
110       if len(args) < 2 {
111         conn.WriteJSON(map[string]interface{}{"success": false, "message":
      "Usage: rm <file>"})
112         return
113       }
114       err := ftpClient.Delete(args[1])
115       if err != nil {
116         sendError(conn, "Failed to remove file", err.Error())
117         return
```

```
118     }
119     sendDirectoryListing(conn, "File removed")
120
121   case "mkdir":
122     if len(args) < 2 {
123       conn.WriteJSON(map[string]interface{}{"success": false, "message":
        "Usage: mkdir <directory>"})
124       return
125     }
126     err := ftpClient.MakeDir(args[1])
127     if err != nil {
128       sendError(conn, "Failed to create directory", err.Error())
129       return
130     }
131     sendDirectoryListing(conn, "Directory created")
132
133   case "rmdir":
134     if len(args) < 2 {
135       conn.WriteJSON(map[string]interface{}{"success": false, "message":
        "Usage: rmdir <directory>"})
136       return
137     }
138     err := ftpClient.RemoveDir(args[1])
139     if err != nil {
140       sendError(conn, "Failed to remove directory", err.Error())
141       return
142     }
143     sendDirectoryListing(conn, "Directory removed")
144
145   default:
146     sendError(conn, "Unknown command", "")
147   }
148 }
149
150 // Function to send directory listing after an operation
151 func sendDirectoryListing(conn *websocket.Conn, message string) {
152   if ftpClient == nil {
153     sendError(conn, "Not connected to any FTP server", "")
154     return
155   }
156
157   entries, err := ftpClient.List("")
158   if err != nil {
159     sendError(conn, "Failed to list directory", err.Error())
160     return
161   }
```

```
162
163   files := []map[string]string{}
164   for _, entry := range entries {
165     files = append(files, map[string]string{
166       "name": entry.Name,
167       "type": func() string {
168         if entry.Type == ftp.EntryTypeFile {
169           return "file"
170         }
171         return "directory"
172       }(),
173     })
174   }
175
176   log.Println(message)
177
178   conn.WriteJSON(map[string]interface{}{
179     "success": true,
180     "files":   files,
181   })
182 }
183
184 // Helper function to send error messages via WebSocket
185 func sendError(conn *websocket.Conn, message, details string) {
186   conn.WriteJSON(map[string]interface{}{
187     "success": false,
188     "message": message,
189     "details": details,
190   })
191 }
192
193 // Helper function to send success messages via WebSocket
194 func sendMessage(conn *websocket.Conn, message string) {
195   conn.WriteJSON(map[string]interface{}{
196     "success": true,
197     "message": message,
198   })
199 }
```

Листинг 2: Реализация файла index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0
    ">
```

```
    <title>FTP Client</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f9;
            margin: 0;
            padding: 0;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }
        .container {
            background-color: #ffffff;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
            width: 300px;
            text-align: center;
        }
        h1 {
            color: #333;
        }
        input[type="text"], input[type="password"] {
            width: 100%;
            padding: 10px;
            margin: 10px 0;
            border: 1px solid #ccc;
            border-radius: 4px;
            font-size: 14px;
        }
        button {
            width: 100%;
            padding: 10px;
            background-color: #0056b3;
            color: #fff;
            border: none;
            border-radius: 4px;
            cursor: pointer;
            font-size: 16px;
        }
        button:hover {
            background-color: #003f87;
        }
        #error-message {
            margin-top: 10px;
```

```
52              color: red;
53          }
54      </style>
55  </head>
56  <body>
57      <div class="container">
58          <h1>FTP Client</h1>
59          <form id="connect-form">
60              <input type="text" id="server" name="server" placeholder="
    FTP Server" required>
61              <input type="text" id="username" name="username" placeholder
    ="FTP Username" required>
62              <input type="password" id="password" name="password"
    placeholder="FTP Password" required>
63              <button type="submit">Connect</button>
64          </form>
65          <p id="error-message"></p>
66      </div>
67      <script>
68          const wsUrl = "ws://185.104.251.226:5421/ws";
69          let ws;
70
71          function initializeWebSocket() {
72              ws = new WebSocket(wsUrl);
73
74              ws.onmessage = function(event) {
75                  const response = JSON.parse(event.data);
76
77                  if (response.success) {
78                      localStorage.setItem("server", document.
    getElementById("server").value);
79                      localStorage.setItem("username", document.
    getElementById("username").value);
80                      localStorage.setItem("password", document.
    getElementById("password").value);
81                      window.location.href = "console.html";
82                  } else {
83                      const errorMessage = document.getElementById("error-
    message");
84                      errorMessage.textContent = `Error: ${response.
    message}`;
85                  }
86              };
87
88              ws.onerror = function() {
```

```
89                  document.getElementById("error-message").textContent = "
    Error: Unable to connect to the server.";
90              };
91
92              ws.onclose = function() {
93                  document.getElementById("error-message").textContent = "
    Connection closed. Please try again later.";
94              };
95          }
96
97          document.getElementById("connect-form").addEventListener("submit
    ", function(event) {
98              event.preventDefault();
99
100             if (ws.readyState !== WebSocket.OPEN) {
101                 initializeWebSocket();
102             }
103
104             const server = document.getElementById("server").value;
105             const username = document.getElementById("username").value;
106             const password = document.getElementById("password").value;
107
108             ws.send(JSON.stringify({
109                 command: "connect",
110                 server,
111                 username,
112                 password
113             }));
114         });
115
116         window.addEventListener("load", initializeWebSocket);
117     </script>
118 </body>
119 </html>
```

Листинг 3: Реализация файла console.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0
    ">
6     <title>FTP Console</title>
7     <style>
8         body {
9             font-family: Arial, sans-serif;
```

```
            background-color: #f4f4f9;
            margin: 0;
            padding: 0;
        }
        .container {
            margin: 20px auto;
            max-width: 800px;
            padding: 20px;
            background-color: #ffffff;
            border-radius: 8px;
            box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
        }
        h1 {
            text-align: center;
            color: #333;
        }
        form {
            margin-bottom: 20px;
            display: flex;
            gap: 10px;
        }
        input[type="text"] {
            flex-grow: 1;
            padding: 10px;
            border: 1px solid #ccc;
            border-radius: 4px;
            font-size: 14px;
        }
        button {
            padding: 10px 15px;
            background-color: #0056b3;
            color: #fff;
            border: none;
            border-radius: 4px;
            cursor: pointer;
            font-size: 14px;
        }
        button:hover {
            background-color: #003f87;
        }
        ul {
            list-style: none;
            padding: 0;
        }
        li {
            padding: 8px 0;
```

```
            border-bottom: 1px solid #e0e0e0;
        }
        li:last-child {
            border-bottom: none;
        }
        .directory {
            font-weight: bold;
            color: #0056b3;
        }
        .file {
            color: #555;
        }
        pre {
            background-color: #f9f9f9;
            padding: 10px;
            border-radius: 4px;
            overflow-x: auto;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>FTP Console</h1>
        <p id="connection-info"></p>
        <form id="command-form">
            <input type="text" id="command" placeholder="Enter command (e.g., ls, mkdir <dir>, rm <file>)">
            <button type="submit">Execute</button>
        </form>
        <h2>Directory Listing</h2>
        <ul id="directory-list"></ul>
        <h2>Output</h2>
        <pre id="output"></pre>
    </div>
    <script>
        // Establish WebSocket connection
        const ws = new WebSocket("ws://185.104.251.226:5421/ws");

        // Retrieve server and username from local storage
        const server = localStorage.getItem("server");
        const username = localStorage.getItem("username");

        // Display connection information
        document.getElementById("connection-info").textContent = `Connected to: ${server} as ${username}`;

```

```
100         // Handle form submission
101         document.getElementById("command-form").addEventListener("submit
    ", function(event) {
102             event.preventDefault(); // Prevent page reload
103             const command = document.getElementById("command").value;
104 
105             if (command.trim() === "") {
106                 alert("Command cannot be empty!");
107                 return;
108             }
109 
110             // Send command to WebSocket server
111             ws.send(JSON.stringify({ command }));
112             document.getElementById("command").value = ""; // Clear
    input field
113         });
114 
115         // Handle incoming messages from WebSocket
116         ws.onmessage = function(event) {
117             const data = JSON.parse(event.data);
118             const output = document.getElementById("output");
119 
120             if (data.files) {
121                 displayDirectory(data.files); // Update directory
    listing
122             } else {
123                 // Display other server responses
124                 output.textContent = JSON.stringify(data, null, 2);
125             }
126         };
127 
128         // Handle WebSocket connection errors
129         ws.onerror = function(event) {
130             alert("An error occurred with the WebSocket connection.");
131         };
132 
133         // Handle WebSocket closure
134         ws.onclose = function() {
135             alert("WebSocket connection closed.");
136         };
137 
138         // Display directory structure
139         function displayDirectory(files) {
140             const directoryList = document.getElementById("directory-
    list");
```

```
141             directoryList.innerHTML = ""; // Clear previous directory
    listing
142
143             files.forEach(file => {
144                 const li = document.createElement("li");
145                 li.textContent = file.type === "directory" ? `        ${
    file.name}` : `        ${file.name}`;
146                 li.className = file.type === "directory" ? "directory" :
     "file";
147                 directoryList.appendChild(li);
148             });
149         }
150     </script>
151 </body>
152 </html>
```

Результат запуска представлен на рисунках 1– 3.

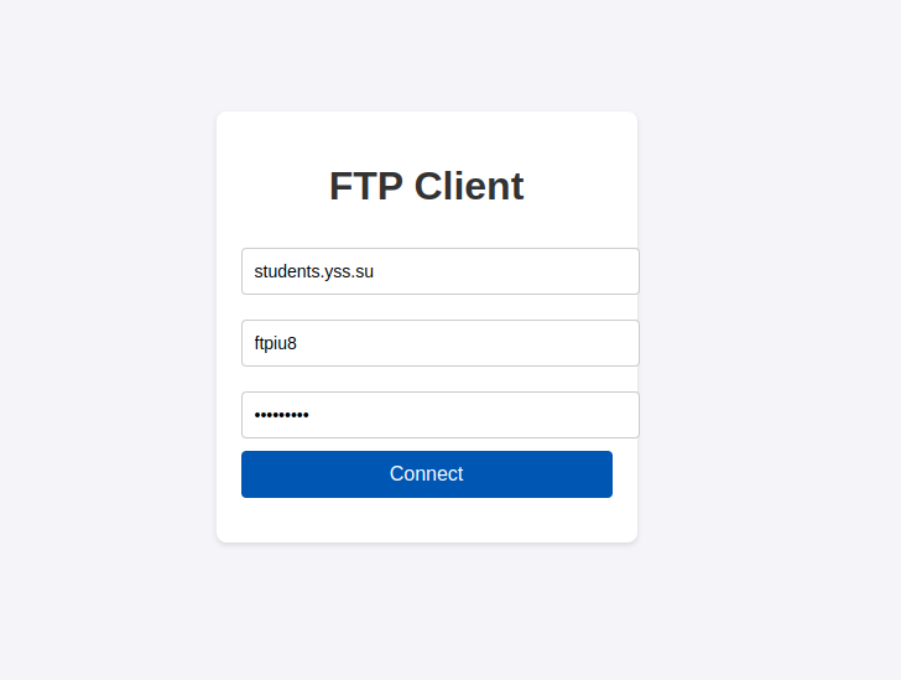

Рис. 1 — Результат со стороны формы для подключения к серверу

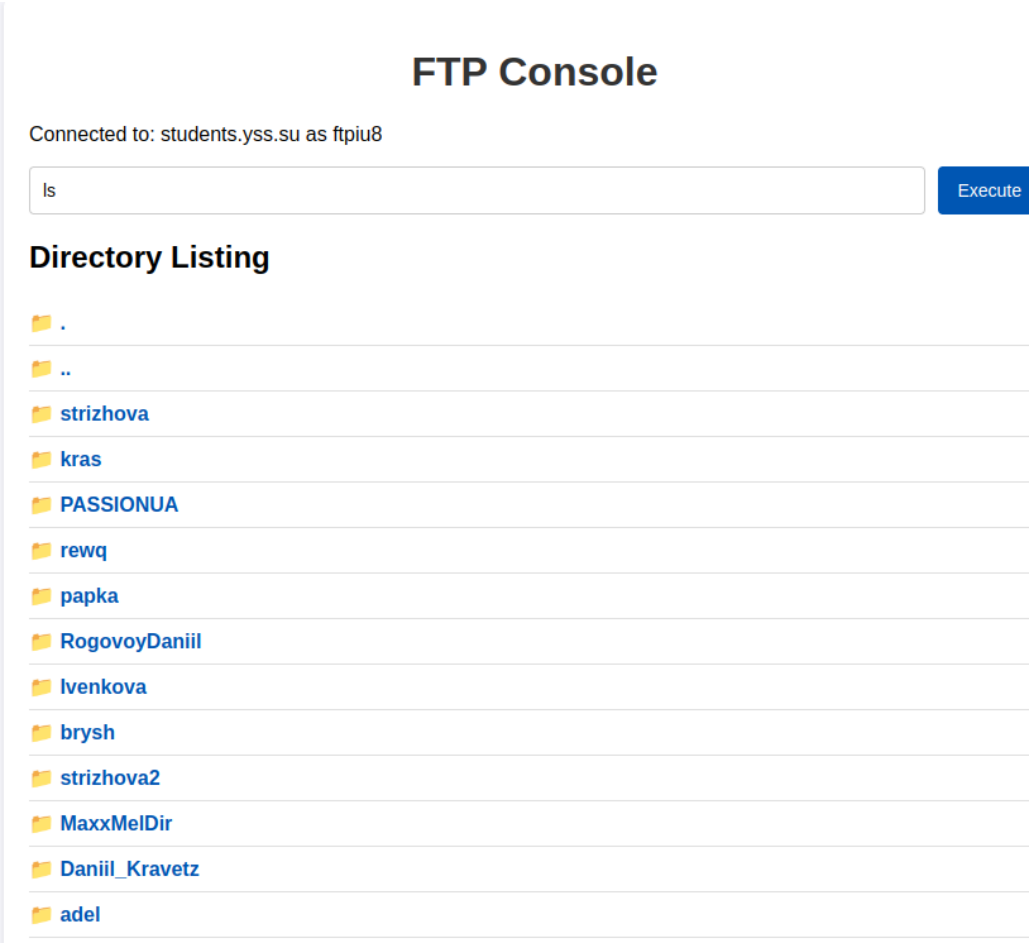

Рис. 2 — Результат со стороны формы для отправки команд

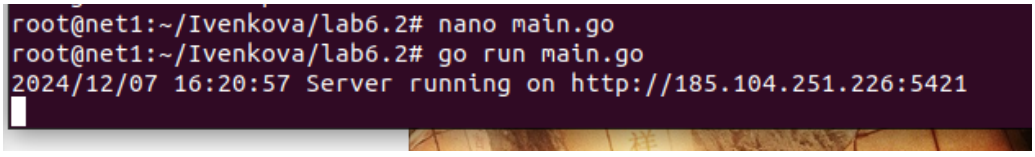

Рис. 3 — Результат со стороны терминала